
AWS Student Portal

Analytics Project

A Complete Cloud-Based Serverless Application with Automated
Analytics

Cloud Computing Project Report

June 29, 2026

AWS Services Used: S3 • API Gateway • Lambda • DynamoDB •
EventBridge • IAM • Power BI

Contents

1	Project Overview	4
1.1	Project Objectives	4
2	Overall Architecture	4
2.1	Architecture Diagram	5
2.2	End-to-End Data Flow Summary	5
3	Step-by-Step Implementation	6
3.1	Step 1: Frontend Development	6
3.1.1	Pages	6
3.1.2	Technologies Used	6
3.2	Step 2: Hosting the Website on Amazon S3	6
3.2.1	How It Works	6
3.2.2	Why S3 for Hosting?	6
3.3	Step 3: API Layer — Amazon API Gateway	7
3.3.1	Role of API Gateway	7
3.4	Step 4: Register Lambda	7
3.4.1	Input (from API Gateway)	7
3.4.2	Processing Steps	7
3.5	Step 5: Login Lambda	8
3.5.1	Processing Steps	8
3.6	Step 6: Amazon DynamoDB — Student Database	8
3.6.1	Table Design	8
3.6.2	Why DynamoDB?	9
3.7	Step 7: The Analytics Bridge Problem	9
3.7.1	The Problem	9
3.7.2	The Solution	9
3.8	Step 8: ExportStudentsToCSV Lambda	9
3.8.1	Internal Logic	9
3.8.2	Overwrite Strategy	10
3.8.3	Why CSV?	10
3.9	Step 9: IAM Permissions	10
3.9.1	Problem Encountered	10
3.9.2	Solution	10
3.10	Step 10: Automation with Amazon EventBridge	11
3.10.1	Configuration	11
3.10.2	How It Works	11
3.11	Step 11: S3 Analytics Bucket	11
3.11.1	Bucket Contents	11
3.12	Step 12: S3 Public Access Configuration	12
3.12.1	Steps Taken	12
3.12.2	Access URL Format	12
3.13	Step 13: Connecting Power BI	12
3.13.1	Connection Steps	12

3.14 Step 14: Power BI Dashboard	12
3.14.1 KPIs and Visuals	13
4 AWS Services Summary	13
5 Architecture Decision Rationale	14
6 Key Concepts Learned	14
7 Current Limitations and Production Enhancements	14
8 Why This Project is Strong for Interviews	15
9 Conclusion	16

1. Project Overview

The **AWS Student Portal Analytics Project** is a fully serverless, cloud-native web application that allows students to register, log in, and have their data automatically exported and visualised through an interactive Power BI dashboard.

The core idea is to eliminate manual data handling. Instead of an administrator periodically downloading data and refreshing reports, the system continuously and automatically keeps the analytics up to date every five minutes using AWS-native automation.

1.1. Project Objectives

- Allow students to **register** and **log in** through a web portal.
- Store all student information **securely in the cloud** using Amazon DynamoDB.
- **Automatically export** student data to a CSV file on a scheduled basis.
- Enable **Power BI** to consume the exported CSV for real-time analytics.
- Achieve complete **end-to-end automation** using serverless AWS services — no manual steps required after initial setup.

Why Serverless?

A serverless approach means there are no servers to provision, patch, or manage. You pay only for what you use, the system scales automatically under load, and the operational overhead is minimal — making it ideal for student projects and production-grade applications alike.

2. Overall Architecture

The system follows a clean, layered architecture that separates concerns clearly across frontend, API, compute, data storage, automation, and analytics.

2.1. Architecture Diagram

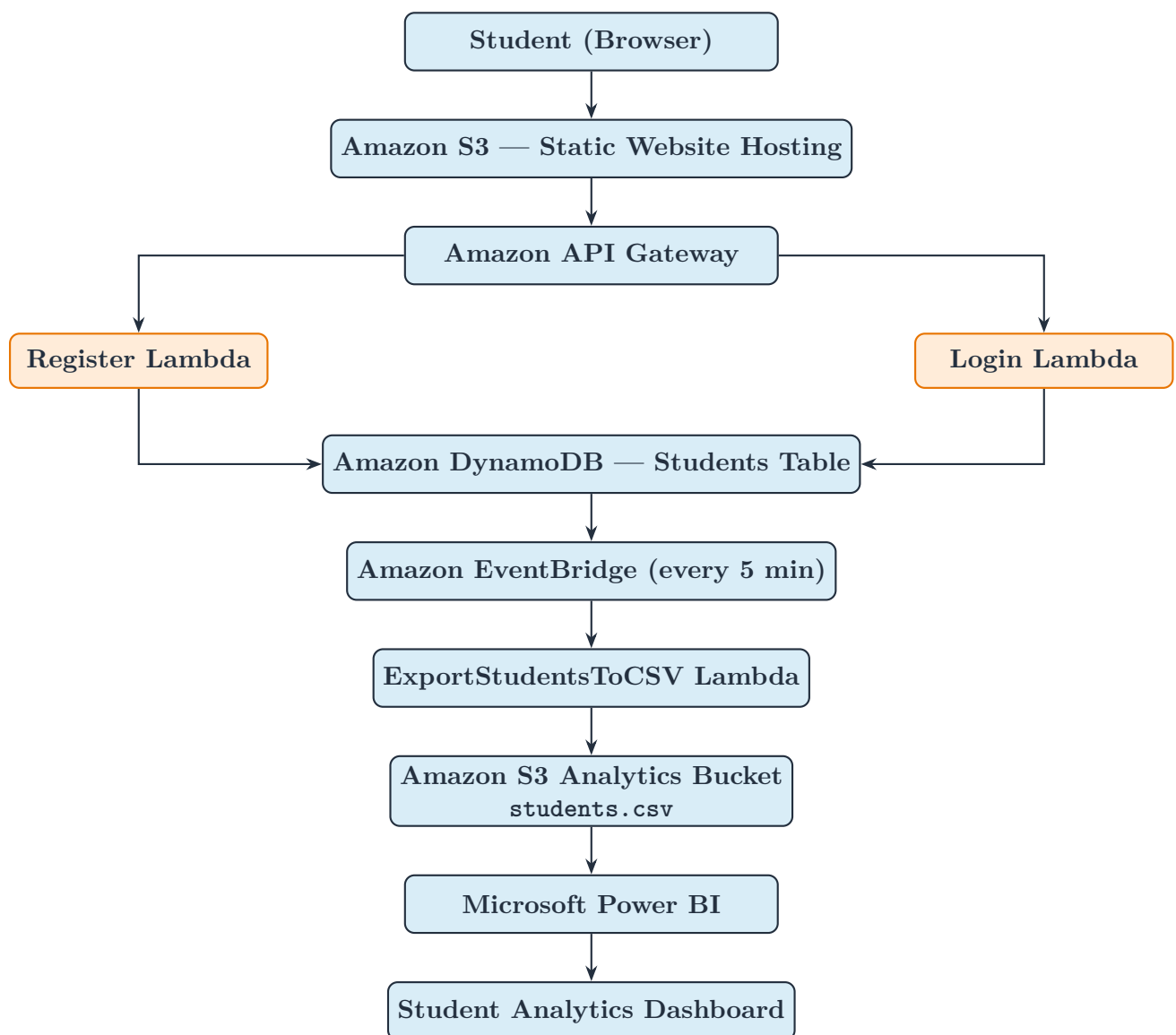


Figure 1: End-to-end architecture of the AWS Student Portal Analytics system

2.2. End-to-End Data Flow Summary

1. A student opens the portal, which is hosted on **Amazon S3**.
2. The student fills in the registration or login form (HTML/CSS/JS frontend).
3. The browser sends an HTTP request to **Amazon API Gateway**.
4. API Gateway routes the request to the appropriate **AWS Lambda** function.
5. Lambda validates the input and reads/writes data in **Amazon DynamoDB**.
6. Every five minutes, **Amazon EventBridge** triggers the **ExportStudentsToCSV Lambda**.
7. This Lambda scans DynamoDB, generates a CSV file, and uploads it to an **S3 analytics bucket**.

8. **Power BI** downloads the CSV via a public URL and refreshes the dashboard automatically.

3. Step-by-Step Implementation

3.1. Step 1: Frontend Development

The student-facing interface is a multi-page web application built with standard web technologies.

3.1.1. Pages

- **Home** — Landing page introducing the portal.
- **Register** — Form for new student registration.
- **Login** — Authentication form for existing students.
- **Dashboard** — Personalised student home screen after login.
- **Courses** — Browse available courses.

3.1.2. Technologies Used

Technology	Role
HTML5	Page structure and semantic markup
CSS3	Styling, layout, and responsive design
JavaScript (ES6)	Form handling, API calls, DOM interaction

The frontend is entirely static — it contains no server-side code. All dynamic behaviour is handled by calling backend APIs through JavaScript's `fetch()` API.

3.2. Step 2: Hosting the Website on Amazon S3

Rather than provisioning and managing a web server, the frontend is deployed to **Amazon S3** using its static website hosting feature.

3.2.1. How It Works

1. An S3 bucket is created and configured for static website hosting.
2. All HTML, CSS, and JavaScript files are uploaded to the bucket.
3. S3 generates a public website URL that users can access in any browser.

3.2.2. Why S3 for Hosting?

- **Cost-effective** — Pay only for storage and requests; typically fractions of a cent per month for small projects.

- **Highly available** — AWS guarantees 99.99% availability for S3.
- **Zero server management** — No EC2 instance to patch or restart.
- **Simple deployment** — Upload files via the AWS Console, CLI, or CI/CD pipeline.

3.3. Step 3: API Layer — Amazon API Gateway

The frontend cannot communicate directly with DynamoDB for two reasons: security (DynamoDB should not be publicly exposed) and architecture (business logic must be enforced). Amazon API Gateway solves both.

3.3.1. Role of API Gateway

API Gateway acts as the **entry point** for all backend requests. It:

- Exposes HTTP endpoints (e.g., `POST /register`, `POST /login`).
- Validates incoming requests.
- Invokes the corresponding Lambda function.
- Returns the Lambda response to the browser.

When a student clicks **Register** or **Login**, the JavaScript sends a `fetch()` request to an API Gateway URL, which in turn triggers the relevant Lambda.

3.4. Step 4: Register Lambda

The `RegisterStudent` Lambda function handles new student sign-ups.

3.4.1. Input (from API Gateway)

- `fullName` — Student's full name.
- `email` — Email address (used as a unique identifier).
- `password` — Account password.
- `city` — City of residence.
- `course` — Enrolled course name.
- `age` — Student's age.
- `learningGoal` — What the student aims to achieve.

3.4.2. Processing Steps

1. Validate that all required fields are present.
2. Generate a unique `studentId` (e.g., using `uuid` or a timestamp-based key).
3. Write the student record into the DynamoDB `Students` table.
4. Return a success or error response.

```
1 POST /register
2 {
3   "studentId":    "s-20240601-001",
4   "fullName":     "Aisha Rahman",
5   "email":        "aisha@example.com",
6   "password":     "hashed_password_here",
7   "city":         "Kozhikode",
8   "course":       "Cloud Computing",
9   "age":          21,
10  "learningGoal": "Become an AWS Solutions Architect"
11 }
```

Listing 1: Example DynamoDB PUT item

3.5. Step 5: Login Lambda

The `LoginStudent` Lambda function authenticates returning students.

3.5.1. Processing Steps

1. Receive **email** and **password** from API Gateway.
2. Query DynamoDB for a student record with the matching email.
3. Compare the submitted password with the stored value.
4. Return **Login Successful** or **Invalid Credentials** accordingly.

Security Note

The current implementation stores passwords in plain text. In a production system, passwords must be hashed using a secure algorithm such as `bcrypt` before storage, and the hash must be compared at login time — never the raw password.

3.6. Step 6: Amazon DynamoDB — Student Database

DynamoDB is the primary data store for all student records.

3.6.1. Table Design

Attribute	Type	Description
<code>studentId</code>	String (PK)	Unique identifier for each student
<code>fullName</code>	String	Student's full name
<code>email</code>	String	Email address
<code>password</code>	String	Hashed account password
<code>age</code>	Number	Student's age
<code>city</code>	String	City of residence
<code>course</code>	String	Enrolled course
<code>learningGoal</code>	String	Student's stated learning objective

3.6.2. Why DynamoDB?

- **Serverless** — No database instance to manage or scale manually.
- **Auto-scaling** — Handles traffic spikes without configuration.
- **High availability** — Data is replicated across multiple AWS Availability Zones.
- **Fast lookups** — Primary key queries return results in single-digit milliseconds.
- **Flexible schema** — Attributes can be added to new records without altering a fixed schema.

3.7. Step 7: The Analytics Bridge Problem

Power BI cannot query DynamoDB directly in an efficient or cost-effective way. A bridging strategy is needed.

3.7.1. The Problem

- DynamoDB does not expose a standard SQL/JDBC interface that Power BI can consume natively.
- Giving Power BI direct DynamoDB access would require AWS credentials in Power BI — a security risk.
- Full table scans from an external BI tool at high frequency would be slow and expensive.

3.7.2. The Solution

DynamoDB → CSV file (in S3) → Power BI

A Lambda function periodically exports the entire DynamoDB table to a CSV file, which is stored in S3 as a public URL. Power BI connects to this URL and refreshes the data automatically.

3.8. Step 8: ExportStudentsToCSV Lambda

This Lambda function is the analytical backbone of the system.

3.8.1. Internal Logic

1. Call `table.scan()` using `boto3` to retrieve all records from DynamoDB.
2. Use Python's `csv` module and `io.StringIO` to build the CSV content in memory.
3. Call `s3.put_object()` to upload the CSV to the S3 analytics bucket as `students.csv`.

```
1 import boto3, csv, io
2
3 def lambda_handler(event, context):
4     dynamodb = boto3.resource('dynamodb')
5     table = dynamodb.Table('Students')
6     response = table.scan()
```

```
7     students = response['Items']
8
9     buffer = io.StringIO()
10    fields =
11        ['studentId','fullName','email','age','city','course','learningGoal']
12    writer = csv.DictWriter(buffer, fieldnames=fields,
13        extrasaction='ignore')
14    writer.writeheader()
15    writer.writerows(students)
16
17    s3 = boto3.client('s3')
18    s3.put_object(
19        Bucket='studentportal-analytics-sahal',
20        Key='students.csv',
21        Body=buffer.getvalue(),
22        ContentType='text/csv'
23    )
24    return {'statusCode': 200, 'body': 'Export successful'}
```

Listing 2: Simplified Python logic for the export Lambda

3.8.2. Overwrite Strategy

The Lambda **always overwrites** `students.csv` rather than creating timestamped files. This ensures Power BI always reads the most current data without needing to manage multiple file versions.

3.8.3. Why CSV?

- Power BI supports CSV natively via Web connector.
- CSV files are lightweight and fast to generate and download.
- No proprietary database driver or AWS connector is required in Power BI.
- The file can be refreshed easily by simply re-running the Lambda.

3.9. Step 9: IAM Permissions

Without the correct IAM permissions, the Lambda functions will fail with `AccessDeniedException` errors.

3.9.1. Problem Encountered

When the export Lambda was first tested, it returned:

```
AccessDeniedException: User is not authorized to perform: dynamodb:Scan
```

3.9.2. Solution

The following AWS managed policies were attached to the Lambda execution role:

Policy	Purpose
AmazonDynamoDBFullAccess	Allows Lambda to scan and query DynamoDB
AmazonS3FullAccess	Allows Lambda to write the CSV to S3

Production Best Practice

Full-access policies grant more permissions than necessary. In production, replace these with least-privilege custom policies — for example, allowing only `dynamodb:Scan` on the specific table and `s3:PutObject` on the specific bucket.

3.10. Step 10: Automation with Amazon EventBridge

Instead of manually triggering the export Lambda, the system uses **Amazon EventBridge Scheduler** to automate it.

3.10.1. Configuration

Setting	Value
Schedule expression	<code>rate(5 minutes)</code>
Target	<code>ExportStudentsToCSV</code> Lambda
Retry policy	2 retries on failure

3.10.2. How It Works

Every five minutes, EventBridge fires an event that invokes the Lambda function. The Lambda then reads DynamoDB and overwrites `students.csv` in S3. This means Power BI data is never more than five minutes stale.

Key Concept

EventBridge Scheduler is serverless — you are not running a cron job on an EC2 instance. There is nothing to manage, and you pay only for the number of invocations triggered.

3.11. Step 11: S3 Analytics Bucket

A dedicated S3 bucket named `studentportal-analytics-sahal` stores the exported CSV.

3.11.1. Bucket Contents

File	Purpose
<code>students.csv</code>	Latest export of all student records (overwritten every 5 minutes)
<code>test.txt</code>	Validation file used during initial setup

Separating analytics storage from the frontend bucket is good practice: it keeps data isolated, makes permission management cleaner, and avoids accidental exposure of web assets.

3.12. Step 12: S3 Public Access Configuration

Power BI accesses the CSV via a standard HTTPS URL. For this to work, the `students.csv` object must be publicly readable.

3.12.1. Steps Taken

1. Disabled *Block Public Access* on the analytics bucket.
2. Attached a bucket policy granting `s3:GetObject` to all principals (*) for the `students.csv` key.
3. Verified the public URL returns the CSV without authentication.

3.12.2. Access URL Format

```
https://<bucket-name>.s3.<region>.amazonaws.com/students.csv
```

Data Privacy Note

Making the CSV publicly accessible is acceptable in this project context. In a real-world deployment, use **S3 pre-signed URLs** or a **CloudFront distribution with signed cookies** to restrict access to authorised users only.

3.13. Step 13: Connecting Power BI

Power BI connects to the live S3 CSV using its built-in Web connector.

3.13.1. Connection Steps

1. Open Power BI Desktop and select **Get Data** → **Web**.
2. Paste the public S3 URL for `students.csv`.
3. Power BI downloads and parses the CSV automatically.
4. Configure a **scheduled refresh** in Power BI Service to pull the latest data periodically.

After connecting, the Power BI data model will contain all fields present in the CSV: `studentId`, `fullName`, `email`, `age`, `city`, `course`, and `learningGoal`.

3.14. Step 14: Power BI Dashboard

With DynamoDB data flowing automatically into Power BI, a rich analytics dashboard can be built.

3.14.1. KPIs and Visuals

Metric	Visual Type	Business Question
Total Students	Card	How many students have registered?
Average Age	Card	What is the typical student age?
Students by Course	Bar / Donut chart	Which courses are most popular?
Students by City	Map / Bar chart	Where are students located?
Learning Goals	Word cloud	What do students want to achieve?
Recent Registrations	Table	Who signed up most recently?
Registration Trend	Line chart	Is enrolment growing over time?
Course Distribution	Pie chart	What is the course mix?

4. AWS Services Summary

Service	Purpose in this Project
Amazon S3 (Frontend)	Hosts the static HTML/CSS/JS student portal website
Amazon API Gateway	Exposes REST API endpoints to the frontend
AWS Lambda	Executes backend logic for registration, login, and CSV export
Amazon DynamoDB	Stores all student records in a serverless NoSQL database
Amazon IAM	Controls permissions for Lambda execution roles
Amazon EventBridge	Triggers the export Lambda automatically every five minutes
Amazon S3 (Analytics)	Stores the exported <code>students.csv</code> file
Microsoft Power BI	Visualises the student data through an interactive dashboard

5. Architecture Decision Rationale

Component	Reason for Choice
S3 Static Hosting	Eliminates the need for a web server; cheap, available, and globally scalable
API Gateway	Decouples the frontend from backend logic; adds a secure, managed API layer
Lambda	Serverless compute that scales to zero when idle; ideal for event-driven workloads
DynamoDB	Fully managed NoSQL; auto-scales for any throughput without database administration
EventBridge	Native AWS scheduler; no infrastructure to run; integrates directly with Lambda
CSV Export	Power BI's most compatible and simplest data source format
Power BI	Industry-standard BI tool with rich visualisation and scheduling support

6. Key Concepts Learned

This project covers a wide range of practical cloud computing skills:

1. **Static website hosting** on Amazon S3.
2. **REST API design and integration** using Amazon API Gateway.
3. **Serverless computing** with AWS Lambda and Python (boto3).
4. **NoSQL database design** using Amazon DynamoDB (partition keys, item attributes, table scans).
5. **IAM roles and policies** for granting least-privilege access between AWS services.
6. **Scheduled automation** using Amazon EventBridge Scheduler.
7. **DynamoDB to CSV export** using Python's `csv` and `io` modules.
8. **S3 file upload** from Lambda using boto3's `put_object` method.
9. **Cloud-hosted data connection** in Power BI via the Web connector.
10. **Automated analytics pipeline** design combining six AWS services and a BI tool.

7. Current Limitations and Production Enhancements

The project is fully functional, but several enhancements would be required before deploying it in a real-world production environment.

Limitation	Recommended Enhancement
Plain-text passwords	Hash passwords using <code>bcrypt</code> or <code>Argon2</code> before storing in DynamoDB
No timestamps	Add <code>registrationDate</code> and <code>lastLogin</code> fields to enable trend and activity analysis
No duplicate email check	Check for existing email before allowing a new registration
Full-access IAM policies	Replace with least-privilege custom policies scoped to specific resources
Full table scan export	Switch to event-driven incremental exports (DynamoDB Streams) as data volume grows
CSV publicly accessible	Use S3 pre-signed URLs or CloudFront signed URLs for secure, time-limited access
No input validation	Sanitise and validate all form inputs on both the frontend and Lambda side
Large data scalability	Consider Amazon Redshift or AWS Glue for datasets that outgrow CSV-based exports

8. Why This Project is Strong for Interviews

This project stands out as a portfolio piece because it demonstrates **end-to-end thinking** across multiple cloud disciplines rather than showcasing a single AWS service in isolation.

What This Project Demonstrates to Interviewers

- Ability to design and build a **complete serverless web application** from scratch.
- Understanding of how to integrate **frontend, backend, database, storage, automation, and BI** into one coherent system.
- Practical experience with **six AWS services** that are widely used in production — S3, API Gateway, Lambda, DynamoDB, EventBridge, and IAM.
- Ability to **solve a real-world reporting challenge** (Power BI cannot read DynamoDB natively) with a pragmatic architectural bridge (scheduled CSV export).
- Awareness of **current limitations** and the ability to articulate a path toward production readiness — a sign of engineering maturity.

9. Conclusion

The AWS Student Portal Analytics Project is a comprehensive, real-world demonstration of modern cloud engineering. By combining a serverless frontend, a managed NoSQL database, event-driven compute, automated scheduling, and business intelligence, the project eliminates manual data operations entirely.

The architecture is cost-efficient (pay-per-use), operationally lightweight (no servers to manage), and automatically scalable. It reflects the same design principles used in production cloud systems at companies worldwide — making it an excellent foundation for further growth into cloud architecture, data engineering, and DevOps roles.

End of Report
